Transformers from Scratch, Architectural Variants, and Byte Latent Transformers: A Controlled Ablation on Binary Cipher Decryption

Vidvathama R

Roll Number: 2024122002

International Institute of Information Technology, Hyderabad

Advanced Natural Language Processing, Assignment 1
September 2, 2026

Abstract

I implement a complete encoder–decoder Transformer from elementary PyTorch operations (no `nn.Transformer`, no `nn.MultiheadAttention`, no `F.scaled_dot_product_attention`) and run a controlled five-way ablation on decrypting XOR-enciphered binary sequences into plaintext English. From a baseline (**C1**), each variant changes *exactly one* component: RoPE, grouped-query attention, RMSNorm, and a token-free Byte Latent Transformer with entropy-based dynamic patching. All five share depth, width, learning rate, batch size, schedule and seed, so every difference is attributable to the component that moved. I report bit and sequence accuracy, Levenshtein distance and BLEU/ROUGE under greedy decoding, alongside throughput and true per-process peak GPU memory.

1 Introduction

The Transformer [14] is usually presented as a monolith, but it is an assembly of independently replaceable parts: a way of injecting position, an attention pattern, a normalisation rule, and a decision about what counts as a token. Each has a well-known modern replacement: RoPE [12], grouped-query attention [1], RMSNorm [16], byte-level processing [7]. Each of them is usually adopted wholesale, bundled with a dozen other changes, which makes it hard to say what any one of them actually bought. This report isolates them: I build every component from scratch and vary one at a time against a fixed baseline, holding all other hyperparameters constant. The contributions are from-scratch implementations of each module, verified by unit tests against reference formulations (Section 3); a controlled five-way ablation (Section 4); and an analysis of the token-free trade-off, covering what the BLT buys and what it costs (Section 6).

2 Task and Data

The corpus. 5,000 strictly line-aligned pairs: `brown_cipher.txt` holds strings over $\{0, 1\}$ and `brown_plain.txt` the corresponding English sentences (53-character alphabet: letters and space). Every plaintext character corresponds to exactly eight cipher bits, so $|c_k| = 8\,|p_k|$ for all 5,000 lines. Plaintext lines average 598 characters (max 2,670), so cipher lines average 4,781 bits (max 21,360).

What the cipher is. Reading the cipher eight bits at a time as a byte and XOR-ing against the aligned plaintext character exposes a constant repeating stream: the eight bytes of the ASCII string `ANLP2026`, restarting at the beginning of every line. The mapping is therefore

$$c_k[8i : 8i{+}8] \;=\; \mathrm{bits}_8\big(\mathrm{ord}(p_k[i]) \oplus K[i \bmod 8]\big), \tag{1}$$

with K = `ANLP2026`. Two consequences matter. First, the task is *exactly* learnable, with no irreducible noise, so any shortfall is attributable to optimisation or capacity, not the data. Second, and more important for the ablation, the correct output at position i depends on i mod 8: the task is intrinsically **position-dependent with period eight**, which is what makes the C1-vs-C2 positional comparison substantive rather than decorative here. *The key is never supplied to any model*; it is recovered only to characterise the task.

Segmentation. A 4,781-token encoder sequence is infeasible under $O(T^2)$ attention on a 4 GB GPU, so I cut each document into aligned segments of 32 plaintext characters (256 cipher bits). Two details keep this honest: segmentation happens *after* the 80/10/10 document-level split, so no document contributes to two splits; and 32 is a multiple of the key period 8, so every segment begins at key phase 0. This yields **76,823 / 9,461 / 9,544** train/val/test segments.

Direction and metrics. Models map cipher to plaintext. Bit accuracy is computed on the 8-bit expansion of predicted and reference *characters*; sequence accuracy requires an exact match of the whole 32-character segment; Levenshtein [4] is the mean edit distance; BLEU [8] and ROUGE [5] are word-level. All generation uses **greedy decoding**, as the brief requires.

3 Architecture

Everything below uses elementary tensor operations. The stack is pre-norm [15].

3.1 Scaled dot-product attention

The core operation is implemented directly as $\mathrm{softmax}(QK^\top/\sqrt{d_k})V$. Masks are boolean, broadcast to $[B, H, T_q, T_k]$. Forbidden logits are filled with `finfo(dtype).min` rather than $-\infty$: a fully-masked row, which genuinely occurs for padded *query* positions, then yields a uniform distribution instead of `NaN`, and those rows are discarded by the loss' `ignore_index`. Using $-\infty$ is a common, hard-to-find source of `NaN` gradients.

3.2 Multi-head and grouped-query attention

MHA projects to H query, key and value heads of width $d_k = d_{\text{model}}/H$. GQA [1] keeps H query heads but only $H_{kv} < H$ key/value heads, each shared by H/H_{kv} query heads; $H_{kv} = H$ recovers MHA exactly and $H_{kv} = 1$ is multi-query attention [11]. Sharing uses `expand` + `reshape`, so nothing is materialised until the matmul consumes it. GQA shrinks the K/V projections and the decoding KV-cache from $2BHTd_k$ to $2BH_{kv}Td_k$. I use $H = 8$, $H_{kv} = 2$; a unit test asserts $H_{kv} = H$ reproduces MHA numerically.

3.3 Positional encoding and normalisation

Sinusoidal absolute encodings (C1, C3–C5) are the fixed table of Vaswani et al. [14], added to the embeddings as a non-trainable buffer. **RoPE** (C2) instead rotates each 2-dimensional slice of every query and key by an angle proportional to its absolute position, so the logit depends only on the relative offset: $\langle R_m q, R_n k\rangle = g(q, k, m-n)$. Because RoPE acts inside attention, C2 adds *no* absolute encoding; otherwise the change would not be a single-component swap. Two implementation points matter: RoPE is applied to the new slice *before* the KV-cache is

concatenated, since cached keys are already rotated; and cross-attention is left un-rotated, query and key living in different sequences where relative distance is meaningless.

LayerNorm [2] re-centres and re-scales; **RMSNorm** [16] drops the mean subtraction, keeping $x/\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon} \odot \gamma$, which is one fewer reduction and no bias vector. Both accumulate statistics in float32 even under bfloat16 autocast, since x^2 summed over a long feature axis underflows badly in half precision.

3.4 Tokenisation and the Byte Latent Transformer

C1–C4 use a byte-level BPE [10] trained from scratch, separately per side, with a 4,096 vocabulary. Because a cipher string is one enormous whitespace-free "word", pre-tokenisation is length-capped at 32 characters rather than split on whitespace. Learned source pieces average 16.8 characters across 29 distinct lengths: genuine variable-length subwords, not a degenerate fixed 8-bit code; a unit test asserts this.

C5 removes the vocabulary entirely: 256 byte values plus three specials, with a standard learned 256-entry byte embedding.

Bytes, not the text of bytes. The ciphertext is a *binary sequence* stored as ASCII `0`/`1` characters, so every eight bits are packed MSB-first into one byte in $[0, 255]$ before the model sees them. Treating each character as its own byte would feed the model the *text encoding* of the data and inflate the sequence eightfold: a 256-character segment is 32 bytes, not 256. The plaintext side is ordinary text, so its UTF-8 bytes are used directly.

Entropy-based dynamic patching. Patches are *not* fixed width. Following BLT, a new patch opens wherever the next byte is hard to predict, so predictable runs are grouped and surprising positions get short patches. The next-byte entropy is estimated by a lightweight **order-2 byte n-gram (Markov) model** with add-α smoothing and stupid backoff, fit by a single counting pass over the training split only. No separate neural language model is trained. A new patch opens at position t when

$$H(x_t \mid x_{t-2}, x_{t-1}) > \theta. \tag{2}$$

Because the estimator is an n-gram, this entropy depends only on the context, so it precomputes into three lookup tables (H_0, $H_1[256]$, $H_2[256^2]$) registered as buffers: training and incremental generation therefore run identical boundary logic. A raw threshold does not transfer between data of different character, so θ is calibrated as the quantile of training entropies giving a target mean patch length. Here that yields 10.3 patches per 34-byte sequence, mean length 3.3 bytes, across 32 distinct lengths: genuinely variable, not fixed width in disguise.

The pipeline. Packed bytes enter a small **local encoder** with windowed attention; a **patch pooler** attention-pools the members of each entropy-defined patch; the **global transformer**, byte-for-byte the same stack C1 uses, runs over the shorter patch sequence; and a **local decoder** with windowed causal self-attention and cross-attention to the patch states emits next-byte logits.

Causality. A learned **start patch** is prepended to the target patch sequence, so the global decoder consumes $[\text{start}, p_1 \dots p_M]$ and emits $[g_0 \dots g_M]$. The target local encoder is causal; p_k pools only its own members; the global decoder is causal over patches. A byte in patch j may attend $g_0 \dots g_j$, and g_j pooled only patches $0 \dots j-1$, that is bytes strictly before patch j

Cfg	Positional	Attention	Norm / Token.
C1	Sinusoidal	MHA	LayerNorm / BPE
C2	**RoPE**	MHA	LayerNorm / BPE
C3	Sinusoidal	**GQA**	LayerNorm / BPE
C4	Sinusoidal	MHA	**RMSNorm** / BPE
C5	Sinusoidal	MHA	LayerNorm / **BLT**

Table 1: The five configurations. Each of C2–C5 changes exactly one component (in **bold**) relative to the C1 baseline.

	Quality (greedy decoding)					Cost			
Cfg	**Bit %**	**Seq %**	**Lev.**	**BLEU**	**R-L**	**Par (M)**	**s/ep**	**Peak MB**	**Dec (s)**
C1	97.17	77.20	0.44	90.34	95.10	7.62	143.0	372	6.6
C2	**97.61**	**81.20**	**0.35**	**92.27**	**96.06**	7.62	153.8	372	6.6
C3	96.81	73.80	0.54	88.54	94.21	6.73	147.2	360	6.1
C4	97.33	78.80	0.42	90.91	95.42	7.61	131.9	350	5.7
C5	97.02	70.00	1.00	82.80	91.23	6.71	275.7	523	27.6

Table 2: Test-set results (25 epochs each); best per column in **bold**. Levenshtein is mean edit distance per segment, lower is better. Cost measured on one RTX 3050 Laptop GPU (4 GB, bfloat16), each configuration in its own process; 'Dec.' decodes 1,000 segments.

begins. Dynamic patching preserves this because the boundary at t is a function of x_{t-2}, x_{t-1} by Eq. 2 and never of x_t, so a byte's patch is known *before* it is generated. The start patch is what makes the first patch work: having cross-attended to the encoder memory, its bytes get full source information with zero target leakage; without it their cross-attention rows would be fully masked, and a fully-masked softmax row is uniform, silently averaging over future patches rather than failing loudly. Unit tests verify all of this numerically.

4 Experimental Setup

All configurations share: $d_{\text{model}} = 256$, 3 encoder + 3 decoder layers, 8 heads, $d_{\text{ff}} = 1024$, dropout 0.1, tied embeddings [9], AdamW [6] ($\beta = 0.9/0.98$, weight decay 0.01), peak LR 3×10^{-4} with 500 warmup steps and cosine decay to 10^{-5}, label smoothing 0.1 [13], gradient clipping 1.0, batch size 64, 25 epochs, seed 42. C3 uses $H_{kv} = 2$; C5 uses local width 128, one local encoder and two local decoder layers, window 64, and a threshold calibrated to a mean patch length of 4 bytes, capped at 12.

Training ran on a single **NVIDIA RTX 3050 Laptop GPU (4 GB)** under bfloat16 autocast, logged to Weights & Biases [3]. Crucially, **each configuration was trained in its own process**: `torch.cuda.max_memory_allocated` is a per-process high-water mark, so running all five together would let C1's peak leak into C5's number. Epoch time is the *median* over epochs, which is robust to host interruptions (one C4 epoch absorbed an OS suspend and recorded 24,890 s against a true ≈127 s).

5 Results

All five configurations ran the full 25 epochs. The baseline C1 reaches 97.17% bit accuracy and reconstructs 77.20% of 32-character segments exactly. Among C1 to C4 the ablation separates

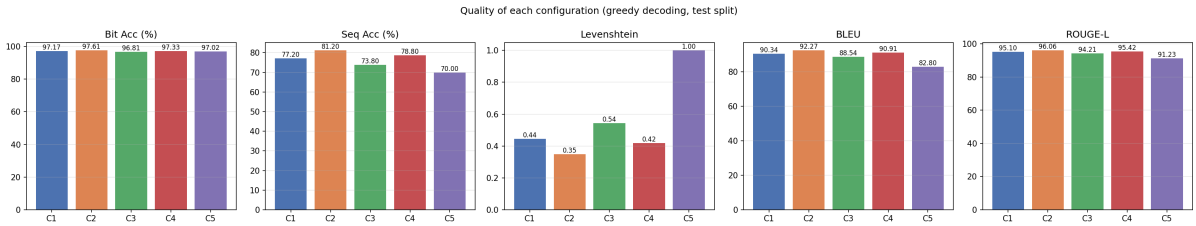


Figure 1: Decoding quality across the five configurations.

good models from slightly better ones, so those differences are small but move consistently across every metric. C1 fails in short local bursts on rare words, turning `Dynasty` into `Dylesty`: one mispredicted BPE token rewrites several characters at once, and since pieces are variable-length it also shifts everything after it. This is why sequence accuracy is far harsher than bit accuracy throughout, as one bad token forfeits the whole segment.

5.1 C2: RoPE improves every quality metric for free

Rotary embeddings give the largest gain among the tokenised models. Sequence accuracy rises from 77.20% to 81.20%, a **4.0-point absolute improvement**, with bit accuracy 97.61%, edit distance 0.35, and BLEU up from 90.34 to 92.27. This costs **zero parameters** (7.62 M, identical to C1) and **zero extra memory** (372 MB against 372 MB): RoPE adds no learnable weights, only a deterministic rotation of q and k.

This fits the task. By Eq. 1 the output at position i depends on i mod 8, so the decoder needs not *where* it is absolutely but *how far* it has advanced relative to the source it consumes, which is what RoPE encodes: the logit is a function of the offset $m - n$ alone. The cost is throughput, 153.8 s per epoch against 143.0 (7.6% slower); decoding is unaffected, as the tables are precomputed.

5.2 C3: GQA trades accuracy for parameters and decode speed

GQA worsens every quality metric: sequence accuracy 73.80% (−3.4 points), bit accuracy 96.81%, edit distance 0.54. Collapsing 8 key/value heads to 2 removes **0.89 M parameters** (−11.6%) and forces four query heads to share one key/value subspace, so the model represents fewer distinct ways of matching queries against keys. Where the decoder must attend to a precisely located 8-bit window, that lost specificity is not free. What GQA buys is inference cost: decoding drops to 6.1 s (−7.6%) and peak memory to 360 MB, as the KV-cache shrinks fourfold.

One detail is easy to misread: C3 is *slower* to train than C1 (147.2 against 143.0 s per epoch) despite having fewer parameters. Under teacher forcing there is no KV-cache to shrink, so only the broadcast of 2 key/value heads back to 8 remains, which is pure overhead. GQA is an *inference-time* optimisation, and this experiment separates the two regimes.

5.3 C4: RMSNorm is a strict improvement

RMSNorm improves quality *and* reduces cost on every axis at once. Sequence accuracy rises to 78.80% (+1.6 over C1) and bit accuracy to 97.33%, while epoch time falls to 131.9 s (7.7% faster, quickest of all five), peak memory to 350 MB (−6.0%), and decoding to 5.7 s (−14.9%). Dropping mean-subtraction removes a full reduction over the feature axis in both passes, and dropping the bias explains the small parameter difference. The accuracy gap is within run-to-run noise (perplexities 1.19 and 1.19), so the honest reading is not “RMSNorm is more accurate” but

"**re-centring contributes nothing measurable here, and its cost is real**." Pre-norm streams are already approximately zero-mean.

6 The token-free trade-off (C5)

C5 is the only configuration that changes the input representation rather than a computation inside the network, and the only one that loses to the baseline. Sequence accuracy falls to 70.00% against C1's 77.20%, a **7.2-point drop**, edit distance more than doubles to 1.00, and BLEU falls to 82.80, yet bit accuracy is essentially unchanged (97.02%). The model gets the bytes right and assembles them badly.

Right bytes, wrong places. C5 has the *lowest* teacher-forced perplexity of all five (1.10 against 1.19) yet the worst sequence accuracy: given the correct prefix the next byte is almost determined, but under free-running decoding it is weakest, so the errors are about *where* a byte belongs rather than *which* one. C5 turns `half of the Mi` into `off hald the M`, the right characters reordered with an insertion pushing the rest out of phase, and one such slip forfeits the segment.

Why entropy patching is a poor fit *here*. By Eq. 1 the task is byte-local and rigidly aligned: after packing, output byte i depends on source byte i and on i mod 8, so the ideal model learns a fixed diagonal correspondence. Entropy patching is instead *content*-driven, and the two sides are cut at unrelated places: cipher bytes are near-uniform ($H_0 = 6.04$ bits, threshold 4.94) so their boundaries are near-arbitrary, while plaintext is English ($H_0 = 4.34$, threshold 3.98) and breaks at word-like positions. Neither respects the key period of 8 nor aligns with the other, so the bottleneck must rebuild a strict one-to-one correspondence across boundaries that do not correspond. This reflects the interaction between mechanism and task, not a defect of the mechanism: on natural language the same procedure is what makes BLT work.

Cost. Packing the bits makes the stream 32 bytes rather than 256, and entropy patching compresses that to about 10.3 patches, so C5's global transformer runs on a *shorter* sequence than C1's 23 BPE tokens. Peak memory is nevertheless 523 MB against 372 MB (1.41×): the saving is outweighed by the byte-level periphery, two extra local stacks over the 34-byte stream plus the dense masks variable-length pooling needs. Epoch time is 275.7 s (1.93×) and decoding 27.6 s (4.15×), as 32 bytes are emitted one at a time instead of roughly 21 tokens. The one clear win is size: C5 has the **fewest parameters** (6.71 M), since dropping the vocabulary deletes two 4096×256 embedding matrices.

7 Conclusion

RoPE buys +4.0 points of sequence accuracy for no parameters and no memory; **GQA** costs 3.4 points and is *slower* to train, its benefit being a KV-cache effect teacher forcing never exercises; **RMSNorm** is free on every axis; and the **token-free BLT** costs 7.2 points at 1.41× memory and 4.15× decode time, with the fewest parameters. The three changes *inside* the network are small and roughly free, while the change to the *representation* dominates, its sign depending on whether it respects the mapping's structure. BPE merges cut across the byte boundaries this cipher is defined on, so C1 makes substitution errors; entropy patches cut the sides at unrelated positions, so C5 makes positional ones. A representation is good or bad only relative to the alignment a task demands.

Links

- **Weights & Biases project (all five runs):**
  `https://wandb.ai/vidvathama-babu-iiit-hyderabad/anlp-a1-blt-ablation`
- **Hugging Face checkpoints (C1–C5):**
  `https://huggingface.co/Viv0605101/anlp-a1-transformer-blt-ablation`

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of EMNLP*, 2023.

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[3] Lukas Biewald. Experiment tracking with weights and biases, 2020. Software available from `https://www.wandb.com/`.

[4] Vladimir I. Levenshtein. Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, 10(8):707–710, 1966.

[5] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[6] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[7] Artidoro Pagnoni, Ram Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason Weston, Luke Zettlemoyer, Gargi Ghosh, Mike Lewis, Ari Holtzman, and Srinivasan Iyer. Byte latent transformer: Patches scale better than tokens. *arXiv preprint arXiv:2412.09871*, 2024.

[8] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Proceedings of ACL*, 2002.

[9] Ofir Press and Lior Wolf. Using the output embedding to improve language models. In *Proceedings of EACL*, 2017.

[10] Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of ACL*, 2016.

[11] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.

[12] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.

[13] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of CVPR*, 2016.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[15] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *Proceedings of ICML*, 2020.

[16] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.